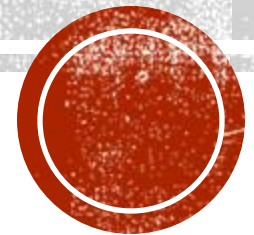


WEEK 5

Multi-class Classification



MULTI CLASS PROBLEM

- Each class has a discriminant function for each class
- The discriminant function for class k will be

$$y_k(\mathbf{x}) = \mathbf{w}_k \mathbf{x} + w_0$$

- Any point x will assign to class k if $y_k(\mathbf{x}) > y_j(\mathbf{x})$ for all $j \neq k$
- The error function becomes

$$E(w) = 0.5 (y_k(x;w) - t_k)^2$$

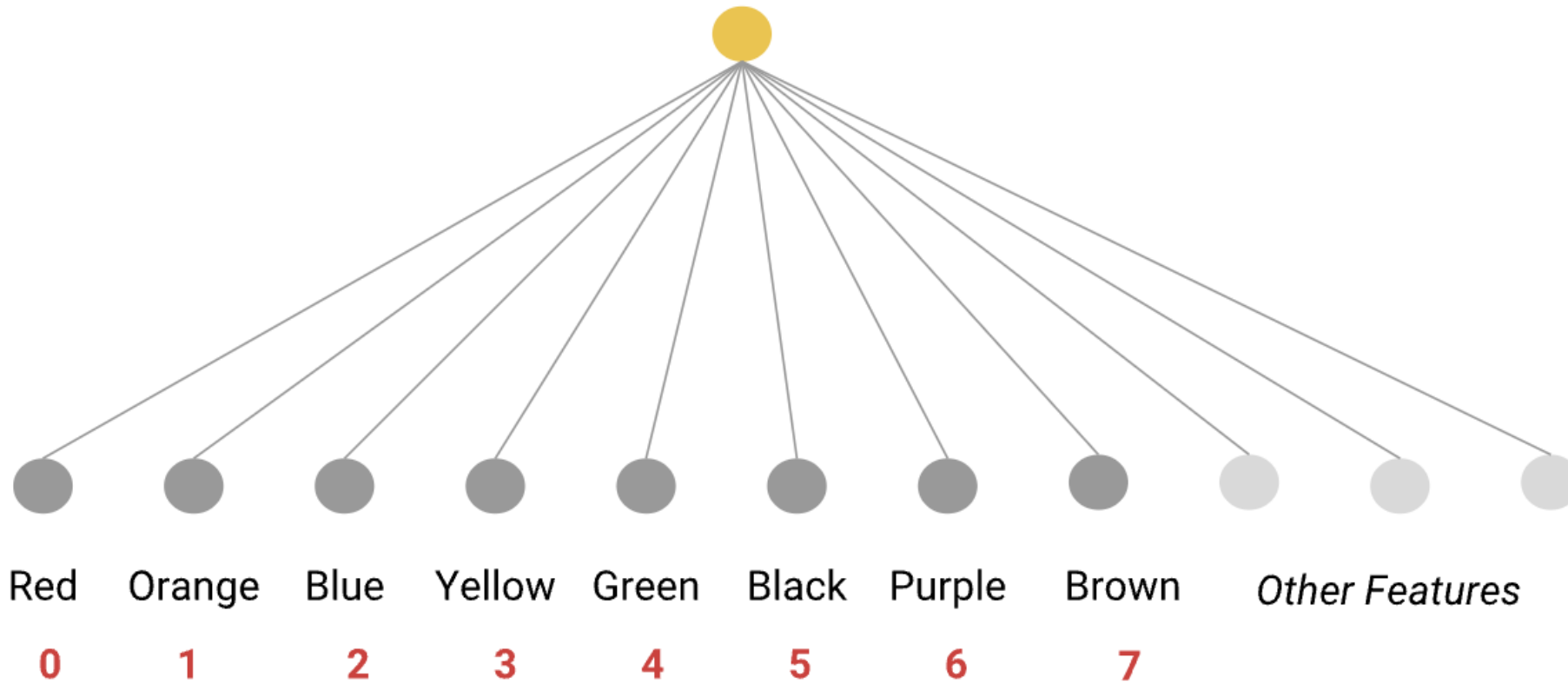


CATEGORICAL DATA: ONE-HOT ENCODING

Feature name	# of categories	Sample categories
snowed_today	2	True, False
skill_level	3	Beginner, Practitioner, Expert
season	4	Winter, Spring, Summer, Autumn
day_of_week	7	Monday, Tuesday, Wednesday
planet	8	Mercury, Venus, Earth



INDEX NUMBER



ONE-HOT CODE- ENCODING

Feature	Red	Orange	Blue	Yellow	Green	Black	Purple	Brown
"Red"	1	0	0	0	0	0	0	0
"Orange"	0	1	0	0	0	0	0	0
"Blue"	0	0	1	0	0	0	0	0
"Yellow"	0	0	0	1	0	0	0	0
"Green"	0	0	0	0	1	0	0	0
"Black"	0	0	0	0	0	1	0	0
"Purple"	0	0	0	0	0	0	1	0
"Brown"	0	0	0	0	0	0	0	1



EXAMPLE OF IRIS DATA

SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3	1.4	0.2	Iris-setosa
6	3.4	4.5	1.6	Iris-versicolor
6.7	3.1	4.7	1.5	Iris-versicolor
6	3.4	4.5	1.6	Iris-versicolor
6.7	3.1	4.7	1.5	Iris-versicolor
6.3	2.3	4.4	1.3	Iris-versicolor



TRANSFORMATION

SepalLength Cm	SepalWidth Cm	PetalLength Cm	PetalWidth Cm	Species	Iris-setosa	Iris- versicolor	Iris- virginica
5.1	3.5	1.4	0.2	Iris-setosa	1	0	0
4.9	3	1.4	0.2	Iris-setosa	1	0	0
6	3.4	4.5	1.6	Iris- versicolor	0	1	0
6.7	3.1	4.7	1.5	Iris- versicolor	0	1	0
6	3.4	4.5	1.6	Iris- virginica	0	0	1
6.7	3.1	4.7	1.5	Iris- virginica	0	0	1
6.3	2.3	4.4	1.3	Iris- virginica	0	0	1



MULTI-CLASS CROSS ENTROPY

- ▶ For recognising c patterns, using *1-of-c* target scheme (c outputs for c patterns/classes)

- ▶ The cross entropy error is:

$$-\sum \sum t \log(p_k)$$

- ▶ Summing the cross entropy across **all network outputs** or prob. outputs together with **number of data**.



TRY CALCULATE CROSS ENTROPY

Example	Iris-setosa	Iris-versicolor	Iris-virginica	P1	P2	p3
1	1	0	0	0.5	0.3	0.2
2	0	1	0	0.2	0.6	0.2
3	0	0	1	0.3	0.4	0.3



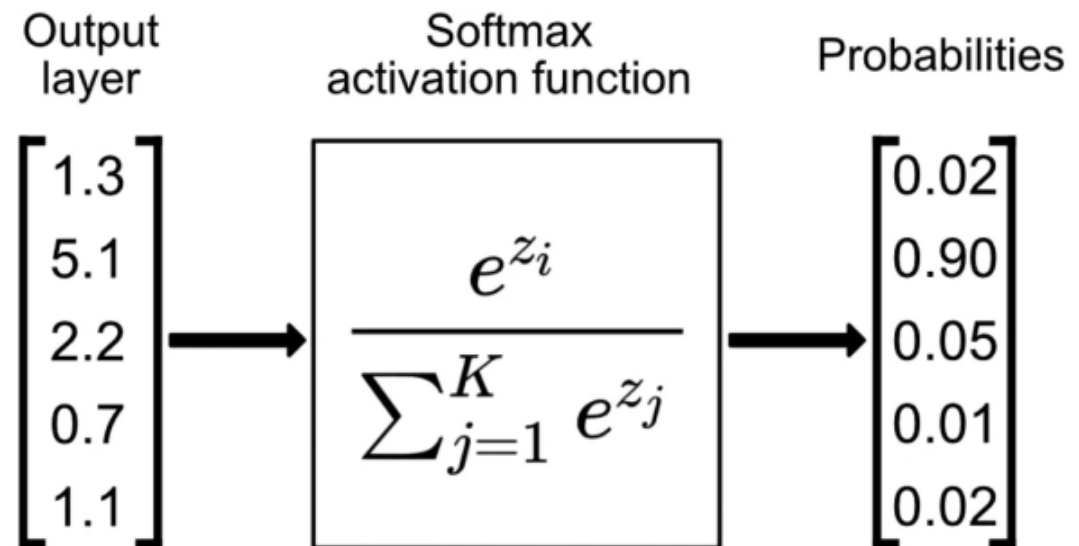
SK-LEARN LIBRARY

- Create
 - `from sklearn.linear_model import LogisticRegression`
 - `model = LogisticRegression(random_state=0)`
- Train
 - `Model=model.fit(X, y)`
- Test
 - `Y_pred=model.predict(X_test)`
- Evaluate
 - `model.score(X_test,y_test)`



OUTPUT ATTRIBUTES

- **predict(X)**
 - Output based on the highest probabilities
- **predict_log_proba(X)**
 - The log probability values
- **predict_proba(X)**
 - Output of each line
 - Softmax is used for normalization



MULTI CLASS CONFUSION MATRIX

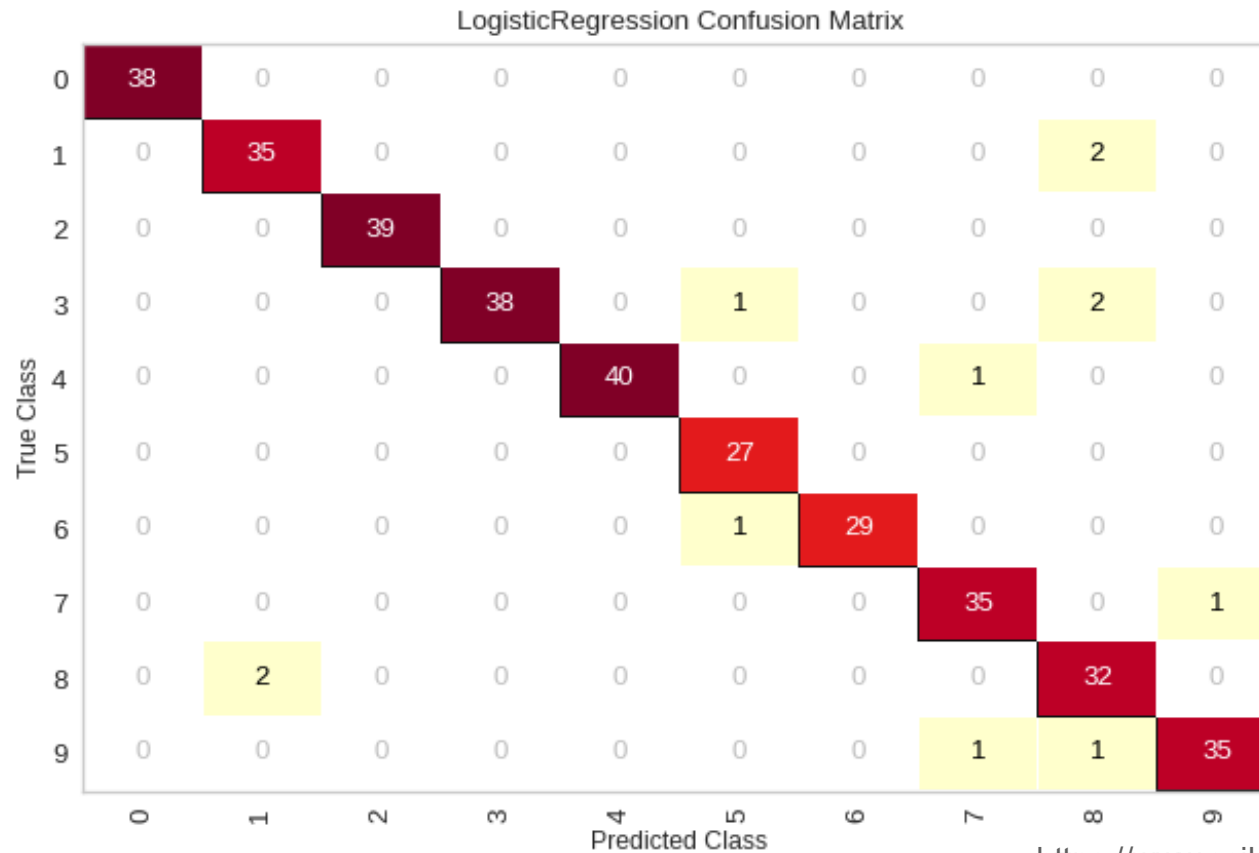
Bigger confusion matrix

- What is the accuracy?
- Recall for **cat**?
- Precision for **cat**?

	Predicted			
		Cat	Dog	Rabbit
Actual	Cat	5	2	0
	Dog	3	3	2
	Rabbit	0	1	11

CONFUSION MATRIX

Just more €



https://www.scikit-yb.org/en/latest/api/classifier/confusion_matrix.html

EXAMPLE

```
from sklearn.metrics import confusion_matrix
y_true = ["cat", "ant", "cat", "cat", "ant", "bird"]
y_pred = ["ant", "ant", "cat", "cat", "ant", "cat"]
matrix=confusion_matrix(y_true, y_pred,
                        labels=["ant", "bird", "cat"])
```



HEATMAP FOR PLOTTING

- **import matplotlib.pyplot as plt**
- **import seaborn as sns**
- **sns.heatmap(matrix,annot=True,cbar=False)**
- **plt.show()**



SKLEARN: FROM MODEL TO CONFUSION_MATRIX

```
from sklearn.metrics import plot_confusion_matrix  
disp = plot_confusion_matrix(model_lg, X, Y,  
                             display_labels=['W', 'M'],  
                             cmap=plt.cm.Blues)  
disp.ax_.set_title("Confusion Matrix")  
  
print("Confusion Matrix")  
print(disp.confusion_matrix)
```

